

네트워크 6 - HTTPS와 TLS 1.3

남궁명수

[1] [HTTPS]

HTTPS(Hypertext Transfer Protocol Secure)는 HTTP 메시지를 TLS로 보호하여 전송하는 통신 방식이다.

HTTP = 평문으로 HTTP 메시지 전송
HTTPS = TLS로 암호화하여 HTTP 메시지 전송

HTTPS는 HTTP와 전혀 다른 메시지 형식을 사용하는 것이 아니다.

클라이언트와 서버는 여전히 HTTP 요청과 응답을 만들지만, 이를 네트워크로 바로 보내지 않고 TLS가 암호화해서 전송한다.

일반적으로 사용하는 포트는 다음과 같다.

HTTP → TCP 80번
HTTPS → TCP 443번

HTTP/1.1과 HTTP/2 기반 HTTPS 통신은 일반적으로 다음 계층으로 구성된다.

HTTP
↓
TLS
↓
TCP
↓
IP

HTTP/3는 TCP 대신 QUIC을 사용하고 TLS 1.3 기능이 QUIC에 통합되어 있다.

HTTP/3
↓
QUIC + TLS 1.3
↓
UDP
↓
IP

[1-1] HTTP의 보안 문제

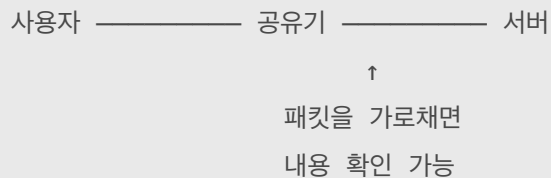
일반 HTTP는 요청과 응답을 암호화하지 않는다.

예를 들어, 로그인 요청을 다음과 같이 보냈다고 하자.

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "id": "myeongsu",
  "password": "1234"
}
```

HTTP에서는 네트워크 중간에서 패킷을 볼 수 있는 공격자가 아이디, 비밀번호, 쿠키와 같은 내용을 읽을 수 있다.



단순히 내용을 읽는 것뿐 아니라 다음 공격도 가능하다.

- 요청이나 응답 내용 변조
- 악성 JavaScript 삽입
- 로그인 세션 쿠키 탈취
- 서버로 위장하는 중간자 공격

[1-2] HTTPS가 제공하는 세 가지 보호

기능	의미
기밀성	제3자가 통신 내용을 읽지 못하게 암호화
무결성	데이터가 전송 중 변조됐는지 검증
인증	통신 상대가 신뢰할 수 있는 서버인지 확인

HTTPS는 단순히 HTTP 내용을 숨기는 기술이 아니다.
데이터 암호화, 변조 탐지, 서버 신원 확인을 함께 제공한다.

[2] [TLS]

TLS(Transport Layer Security)는 네트워크 통신에 다음 기능을 제공하는 보안 프로토콜이다.

- 서버 인증
- 선택적인 클라이언트 인증
- 암호화 키 합의
- 데이터 암호화
- 데이터 무결성 검증

[3] [SSL을 사용하지 않는 이유]

과거에는 SSL(Secure Sockets Layer)이 사용되었지만, 오래된 암호 방식과 구조적인 취약점이 발견되어 폐기되었다.

SSL 2.0 / 3.0 → 오래되고 안전하지 않음

TLS 1.0 / 1.1 → 오래되고 사용 중단

TLS 1.2 / 1.3 → 현재 일반적으로 사용

SSL 3.0에는 대표적으로 POODLE과 같은 취약점이 있다.

구식 암호화 방식으로 다운그레이드(하향 조정)될 때 발생하는 취약점을 이용하는 공격

또한, 현대적인 암호 알고리즘과 TLS 확장 기능을 충분히 지원하지 못한다.

일상적으로는 여전히 “SSL 인증서”, “SSL 설정”이라고 부르지만, 실제 현대 HTTPS 통신에서는 TLS를 사용한다.
인증서 자체는 X.509 인증서이며 이를 TLS 핸드셰이크에서 사용하는 것이다.

[4] [HTTPS에서 사용하는 암호 기술]

HTTPS를 이해하려면 다음 네 가지 개념을 구분해야 한다.

- 대칭키 암호화
- 공개키 암호화
- 해시
- 전자서명

[4-1] 대칭키 암호화

암호화와 복호화에 동일한 비밀키를 사용하는 방식이다.

평문 + 대칭키 → 암호문

암호문 + 같은 대칭키 → 평문

예를 들어:

평문: password=1234

비밀키: K

암호화: Encrypt(K, 평문)

결과: 암호문

대칭키 암호화의 특징은 다음과 같다.

- 처리 속도가 빠름
- 대량 데이터 암호화에 적합
- 송신자와 수신자가 같은 비밀키를 알고 있어야 한다.
- 비밀키를 어떻게 안전하게 공유할 것인지가 중요하다.

HTTPS에서 실제 HTTP 요청과 응답 데이터는 주로 대칭키 방식으로 암호화한다.

대표 알고리즘은 다음과 같다.

- AES-GCM
- ChaCha20-Poly1305

현대 TLS에서 사용하는 위 방식들은 단순 암호화뿐 아니라 데이터 변조 여부를 확인하는 인증 태그도 함께 제공한다.

[4-2] 공개키 기반 기술

공개키 방식에서는 서로 대응하는 두 개의 키를 사용한다.

Public Key → 외부에 공개 가능

Private Key → 서버만 비밀리에 보관

공개키와 개인키는 서로 수직적인 관계를 가지지만 공개키만 보고 개인키를 현실적인 시간 안에 계산하기는 매우 어렵다.

공개키 기술은 크게 다음과 같이 활용할 수 있다.

공개키로 암호화 → 대응하는 개인키로 복호화

개인키로 서명 → 대응하는 공개키로 서명 검증

다만 첫 번째 설명은 RSA 암호화를 이해하기 위한 전형적인 설명이다.

TLS 1.3에서는 서버 공개키로 세션키를 암호화해 보내는 RSA 키 교환을 사용하지 않는다.

TLS 1.3에서 서버 인증서의 공개키와 서버 개인키는 주로 다음 목적으로 사용된다.

- 인증서와 서버 신원 검증
- 서버가 인증서에 대응하는 개인키를 실제로 보유했는지 증명
- 핸드셰이크 내용에 대한 전자서명 생성 및 검증

실제 공유 비밀은 일반적인 ECDHE 키 합의를 통해 만든다.

[4-3] 해시

해시는 입력 데이터를 고정된 길이의 값으로 변환하는 함수다.

원본 데이터 → Hash 함수 → 고정 길이 해시값

해시의 주요 특징은 다음과 같다.

- 같은 입력은 같은 해시값을 생성
- 입력이 조금만 달라져도 해시값이 크게 달라짐
- 해시값에서 원본을 역산하기 어려움
- 데이터 변조 여부 확인에 사용 가능

TLS에서는 해시를 다음 목적으로 사용한다.

- 핸드셰이크 전체 내용의 요약값 계산
- 전자서명 입력 생성
- HKDF를 통한 암호키 파생
- Finished 메시지를 통한 핸드셰이크 무결성 확인

대표적으로 SHA-256과 SHA-384 등이 사용된다.

[4-4] 전자서명

전자서명은 특정 데이터가 개인키 소유자에 의해 만들어졌고, 서명 이후 변경되지 않았는지 확인하는 기술이다.

개념적인 과정은 다음과 같다.

[서버]

데이터 또는 데이터의 해시

↓

서버 개인키로 서명

↓

전자서명 생성

[클라이언트]

서버 공개키로 전자서명 검증

↓

서버가 개인키를 보유하고 있음을 확인

전자서명이 제공하는 주요 성질은 다음과 같다.

- 인증: 해당 개인키를 가진 주체가 서명했음을 확인한다.
- 무결성: 서명 대상 데이터가 변경되지 않았음을 확인한다.
- 부인 방지: 정상적으로 관리된 개인키의 서명을 소유자가 쉽게 부인하기 어렵다.

전자서명은 내용을 숨기는 기술이 아니다.

암호화 → 내용을 숨기기 위한 것

서명 → 발신자와 변조 여부를 확인하기 위한 것

TLS에서는 전자서명이 두 곳에서 중요하게 사용된다.

1. CA가 서버 인증서에 서명한다.
2. 서버가 CertificateVerify 메시지에 서명한다.

이 두 서명은 목적이 서로 다르다.

CA의 인증서 서명

→ 인증서의 도메인과 서버 공개키를 CA가 보증

서버의 CertificateVerify 서명

→ 현재 접속한 서버가 인증서 공개키에 대응하는 개인키를 보유했음을 증명

[5] [왜 대칭키와 공개키 방식을 함께 사용하는가?]

대칭키 암호화는 빠르지만 키 공유 문제가 있다.

클라이언트와 서버가 같은 비밀키를 사용해야 함

그런데 비밀키를 평문으로 보내면 공격자가 탈취 가능

공개키 기반 기술은 인증과 안전한 키 합의에 유리하지만, 대칭키 암호화보다 계산 비용이 크다.
따라서 대량의 HTTP 데이터를 모두 공개키 방식으로 처리하면 비효율적이다.

TLS는 두 방식의 장점을 결합한다.

[핸드셰이크]

인증서와 전자서명 → 서버 인증

ECDHE → 공유 비밀 합의

HKDF → 대칭키 파생

[실제 데이터 통신]

대칭키와 AEAD → HTTP 요청, 응답 암호화 및 무결성 검증

정리하면 다음과 같다.

공개키 기술 → 서버 인증과 전자서명

ECDHE → 공유 비밀 합의

해시·HKDF → 용도별 암호키 파생

대칭키 암호 → 실제 HTTP 데이터 암호화

[6] [TLS 인증서]

TLS 인증서는 서버의 도메인 정보와 공개키를 연결해 주는 전자 문서다.

인증서는 보통 다음 정보를 포함한다.

- 도메인 이름
- 서버 공개키
- 인증서 발급자
- 인증서 유효 기간
- 전자서명 알고리즘
- 인증서 일련번호
- 인증서 용도
- CA의 전자서명

구조를 단순화하면 다음과 같다.

TLS 인증서

```
├ 도메인: example.com
├ 공개키: Server Public Key
├ 발급자: Intermediate CA
├ 유효 기간: 시작일 ~ 만료일
├ 용도: Server Authentication
└ CA 전자서명
```

인증서의 핵심 의미는 다음과 같다.

신뢰할 수 있는 인증기관이 이 공개키가 해당 도메인에 대해 발급된 공개키라는 사실을 인증했다.

인증서는 공개 정보이므로 TLS 핸드셰이크에서 클라이언트에게 전달해도 된다.

반대로 인증서의 공개키와 짝을 이루는 서버 개인키는 절대로 외부에 공개하면 안된다.

[7] [CA와 인증서 발급 과정]

[7-1] CA란?

CA(Certificate Authority)는 인증서를 발급하고 서명하는 신뢰할 수 있는 인증기관이다.

대표적인 인증기관으로는 다음과 같은 곳들이 있다.

- Let's Encrypt
- DigiCert
- GlobalSign
- Sectigo

서버가 단순히 다음과 같이 주장하는 것만으로는 신뢰할 수 없다.

“나는 public-insight.co.kr이고 이 공개키는 내 것이야.”

공격자도 같은 주장을 할 수 있기 때문이다.

따라서 CA가 신청자가 실제로 해당 도메인을 통제하는지 확인하고 인증서에 전자서명한다.

[7-2] public-insight.co.kr 인증서 발급 예시

[1단계: 서버 키 쌍 생성]

서버에서 개인키와 공개키 한 쌍을 생성한다.

public-insight.co.kr 서버
├ Server Private Key → 서버만 비밀로 보관
└ Server Public Key → 인증서에 포함하여 공개

서버 개인키는 CA에도 보내지 않는다.

[2단계: CSR 생성]

서버는 인증서 발급 요청서인 CSR(Certificate Signing Request)을 만든다.

CSR에는 다음과 같은 내용이 포함된다.

도메인: public-insight.co.kr
공개키: Server Public Key
신청자 정보
전자서명: 서버 개인키로 CSR에 서명

구조적으로 보면

도메인 정보 + 서버 공개키
↓
서버 개인키로 서명
↓
CSR

CSR 서명을 통해 CA는 신청자(서버)가 제출한 공개키와 짝을 이루는 개인키를 실제로 보유하고 있는지 확인할 수 있다.

하지만 CSR 서명만으로 신청자가 도메인을 소유했다는 사실 까지 증명되는 것은 아니므로 별도의 도메인 검증이 필요하다.

[3단계: CA의 도메인 통제권 확인]

CA는 신청자가 “public-insight.co.kr”을 실제로 통제하는 지 확인한다.

대표적인 방법은 다음과 같다.

- HTTP 방식: CA가 지정한 값을 웹 서버의 특정 주소에 제공한다.

<http://public-insight.co.kr/.well-known/acme-challenge/검증값>

- CA가 이 주소에 접속해 올바른 값을 확인하면 웹 서버를 통제한다고 판단한다.

- DNS 방식: CA가 지정한 값을 DNS TXT 레코드에 등록한다.

_acme-challenge.public-insight.co.kr → CA가 지정한 검증값

- CA가 DNS 조회를 통해 해당 값을 확인하면 도메인의 DNS를 통제한다고 판단한다.

[4단계: CA의 인증서 생성 및 서명]

도메인 검증이 끝나면 CA가 인증서를 만든다.

인증서 내용

도메인: public-insight.co.kr

서버 공개키: Server Public Key

발급자: Intermediate CA

유효기간: 시작 시각 ~ 만료 시각

일련번호: ...

용도: Server Authentication

실제 공개 인증서는 보통 루트 CA가 아니라 중간 CA가 서명한다.한다.

인증서의 서명 대상 데이터

↓ 해시

인증서 데이터의 해시값

↓ 중간 CA 개인키로 서명

CA 전자서명

최종 인증서는 다음 내용을 포함한다.

[public-insight.co.kr 정보]

[서버 공개키]

[유효기간과 용도]

[발급한 중간 CA 정보]

[중간 CA의 전자서명]

여기서 서버 공개키와 CA 개인키를 혼동하면 안 된다.

서버 공개키

→ 서버 인증서 안에 포함됨

중간 CA 개인키

→ 인증서 서명을 만드는 데만 사용

→ 인증서 안에 포함되지 않음

CA는 완성된 인증서를 서버에 전달한다.

[5단계: 서버에 인증서 설치]

서버에는 일반적으로 다음 파일이 설치된다.

1. public-insight.co.kr 서버 인증서
2. 중간 CA 인증서 체인
3. 서버 인증서 공개키와 짝인 서버 개인키

예를 들어, Nginx 예시는 다음과 같다.

```
ssl_certificate      /etc/ssl/public-insight/fullchain.pem;  
ssl_certificate_key  /etc/ssl/public-insight/private.key;
```

fullchain.pem

- └─ 서버 인증서
- └─ 중간 CA 인증서 또는 인증서들

private.key

- └─ 서버 개인키

[6단계: 사용자의 HTTPS 접속]

```
https://public-insight.co.kr
```

TLS 핸드셰이크 중 서버는 브라우저에 다음 정보를 전달한다.

서버 → 브라우저

- └─ public-insight.co.kr 서버 인증서
- └─ 중간 CA 인증서 체인

서버 공개키와 CA의 서명은 서버 인증서 안에 포함되어 있다.
서버 개인키는 절대로 전송하지 않는다.

[8] [루트 CA와 중간 CA]

[8-1] 인증서 체인

일반적으로 서버 인증서는 루트 CA가 직접 발급하지 않고 중간 CA가 발급한다.

Root CA

↓ 루트 CA 개인키로 중간 CA 인증서에 서명

Intermediate CA

↓ 중간 CA 개인키로 서버 인증서에 서명

public-insight.co.kr Server Certificate

이를 인증서 체인 또는 신뢰 체인이라고 한다.

[8-2] 루트 CA

루트 CA는 인증 체계의 가장 위에 있는 최상위 인증 기관이다.

루트 CA 인증서에는 다음 정보가 들어 있다.

루트 CA 인증서

├ 루트 CA 정보

├ 루트 CA 공개키

└ 루트 CA 자신의 개인키로 만든 서명

루트 인증서는 일반적으로 자체 서명 인증서(Self-signed Certificate)다.

그러면 다음과 같은 의문이 생긴다.

“자기가 자신에게 서명했는데 왜 믿을 수 있을까?”

브라우저와 운영체제의 신뢰 저장소에 신뢰할 수 있는 루트 CA 인증서가 미리 등록되어 있기 때문이다.

브라우저 또는 운영체제의 Root CA Trust Store

├ ISRG Root X1

├ DigiCert 계열 Root CA

├ GlobalSign 계열 Root CA

└ 기타 신뢰받는 Root CA

루트 CA는 다른 전자서명을 따라가서 신뢰를 얻는 대상이 아니라,
브라우저와 운영체제가 정책적으로 미리 신뢰한 신뢰의 출발점이다.

루트 CA 개인키가 유출되면 그 아래의 인증 체계 전체가 위험해질 수 있다.
따라서 루트 CA 개인키는 보통 네트워크와 분리된 환경에 보관하고 제한적으로만 사용한다.

[8-3] 중간 CA

중간 CA는 루트 CA로부터 인증서 발급 권한을 위임받은 인증기관이다.
루트 CA는 중간 CA의 정보와 공개키가 담긴 인증서에 서명한다.

중간 CA 정보 + 중간 CA 공개키
↓ 루트 CA 개인키로 서명
중간 CA 인증서

중간 CA는 자신의 개인키를 실제 서버 인증서에 서명한다.

public-insight.co.kr 정보 + 서버 공개키
↓
중간 CA 개인키로 서명
↓
public-insight.co.kr 인증서

[8-4] 중간 CA를 사용하는 이유

루트 CA가 모든 서버 인증서에 직접 서명하면 루트 개인키를 자주 사용한다.
이는 가장 중요한 개인키의 노출 가능성을 높인다.

중간 CA를 사용하면 다음 장점이 있다.

- 루트 CA 개인키를 오프라인에 안전하게 보관할 수 있다.
- 실제 인증서 발급은 중간 CA가 담당할 수 있다.
- 중간 CA에 사고가 발생하면 해당 중간 CA만 폐기할 수 있다.
- 인증서 종류, 지역, 고객, 용도 등에 따라 권한을 분리할 수 있다.

[8-5] 브라우저에 전달되는 인증서

서버는 일반적으로 다음 인증서를 전달한다.

1. 서버 인증서
2. 중간 CA 인증서 또는 인증서들

루트 인증서는 보통 서버가 전송하지 않는다. 브라우저나 운영체제에 이미 신뢰 저장소에 가지고 있기 때문이다.

브라우저는 아래에서 위로 검증한다.

- ① 서버 인증서의 서명
→ 중간 CA 공개키로 검증
- ② 중간 CA 인증서의 서명
→ 루트 CA 공개키로 검증
- ③ 루트 CA 인증서
→ 브라우저 신뢰 저장소에 존재하는지 확인

모두 성공하면 다음과 같이 판단한다.

- public-insight.co.kr
- 신뢰받는 중간 CA가 보증함
- 그 중간 CA는 신뢰받는 루트 CA가 보증함
- 따라서 서버 인증서를 신뢰할 수 있음

[9] [브라우저의 인증서 검증]

브라우저는 인증서를 받으면 공개키만 꺼내 쓰는 것이 아니라 여러 항목을 검사한다.

[9-1] CA 서명과 체인 검증

서버 인증서의 서명이 중간 CA 공개키로 검증되는지 확인한다.
이어서 중간 CA 인증서가 신뢰하는 루트 CA까지 연결되는지 확인한다.

[9-2] 도메인 이름 검증

현재 접속한 도메인이 인증서의 SAN(Subject Alternative Name)에 포함되어 있는지 확인한다.

접속 주소: www.example.com
인증서 SAN: www.example.com
→ 일치

접속 주소: attacker.com
인증서 SAN: example.com
→ 불일치

현대 인증서에서는 주로 SAN을 이용해 도메인을 확인한다.

와일드카드 인증서는 다음과 같이 사용할 수 있다.

```
*.example.com
```

이는 일반적으로 다음에는 일치한다.

```
api.example.com  
www.example.com
```

하지만 다음처럼 여러 단계 아래의 도메인까지 모두 포함하지는 않는다.

```
v1.api.example.com
```

[9-3] 유효 기간 검증

현재 시간이 인증서의 유효 기간 안에 있는지 확인한다.

```
Not Before ≤ 현재 시간 ≤ Not After
```

인증서가 아직 유효하지 않거나 만료되었다면 경고가 발생한다.

[9-4] 인증서 용도 검증

해당 인증서가 서버 인증 용도로 발급되었는지 확인한다.

예를 들어, Extended Key Usage에 다음과 같은 용도가 포함될 수 있다.

```
TLS Web Server Authentication
```

CA 인증서인지 서버 인증서인지 구분하기 위한 Basic Constraints 등의 제한도 확인한다.

[9-5] 폐기 여부 확인

인증서가 유효기간 안에 있더라도 개인키 유출 등의 이유로 조기에 폐기될 수 있다.

대표적인 확인 방식은 다음과 같다.

- CRL(Certificate Revocation List)
- OCSP(Online Certificate Status Protocol)
- OCSP Stapling

[9-6] 서명 알고리즘과 정책 검증

브라우저는 다음과 같은 내용도 확인한다.

- 허용된 안전한 서명 알고리즘인지
- 인증서 체인의 제약 조건을 지켰는지
- 정책상 신뢰할 수 있는 루트 CA인지
- 인증서의 구조와 서명이 올바른지

[10] [인증서와 서버 개인키의 관계]

서버에는 일반적으로 다음 세가지가 존재한다.

```
Server Certificate
Private Key
Intermediate Certificate Chain
```

인증서에는 서버 공개키가 들어 있지만 서버 개인키는 들어 있지 않다.

```
인증서
└─ 공개키 포함

서버
└─ 개인키 별도 보관
```

개인키가 유출되면 공격자가 해당 서버의 전자서명을 만들 수 있으므로 매우 안전하게 보관해야 한다.
인증서 자체는 공개 정보이므로 공격자도 복사할 수 있다.
따라서 인증서를 전달했다는 사실만으로는 현재 서버가 진짜라고 판단할 수 없다.

서버는 TLS Handshake의 CertificateVerify 메시지를 통해 인증서의 공개키에 대응하는 개인키를 실제로 보유하고 있음을 증명한다.

```
서버
핸드셰이크 내용
    ↓ 서버 개인키로 서명
CertificateVerify

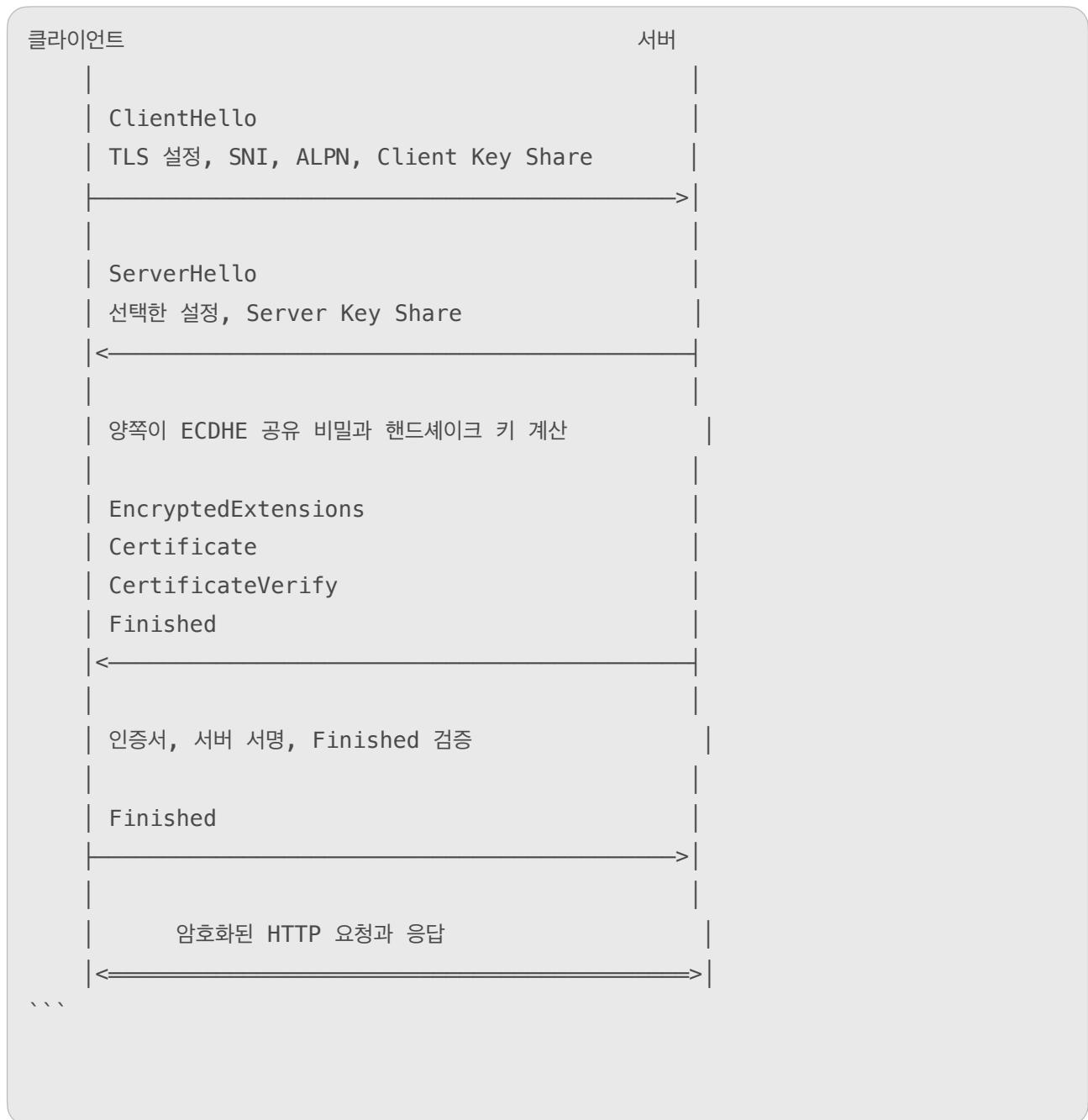
클라이언트
CertificateVerify
    ↓ 인증서의 서버 공개키로 검증
서버의 개인키 보유 확인
```

[11] [TLS 1.3 Handshake의 목적]

TLS Handshake는 HTTPS 데이터를 전송하기 전에 클라이언트와 서버가 다음 작업을 수행하는 과정이다.

- 사용할 TLS 버전 결정
- 사용할 암호화 알고리즘 결정
- 서버 인증서 전달 및 검증
- 서버의 개인키 보유 증명
- 공유 비밀 합의
- 용도와 방향별 암호키 생성
- 핸드셰이크 변조 여부 확인
- TLS 위에서 사용할 HTTP 버전 협상

TLS 1.3의 전체 흐름을 먼저 단순화하면 다음과 같다.



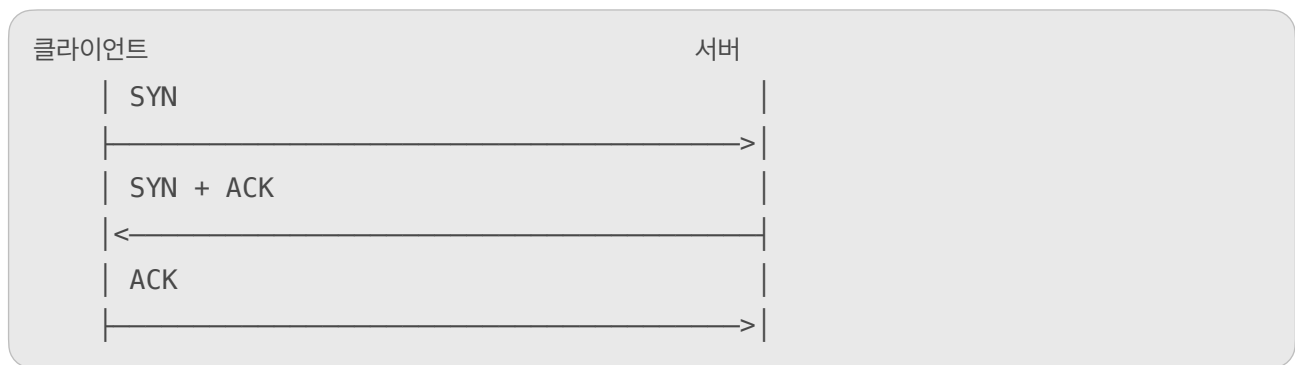
[12] [TLS 1.3 암호화 과정]

HTTP/1.1 또는 HTTP/2 기반 HTTPS에서는 이 TLS Handshake 전에 먼저 TCP 3-Way Handshake가 수행된다.

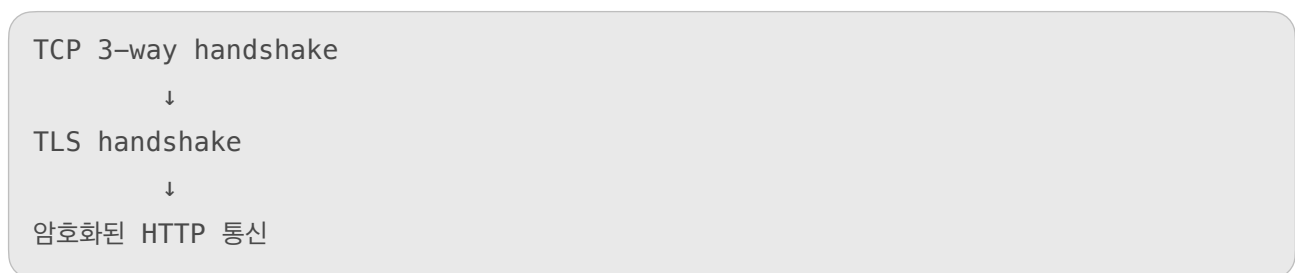
1. TCP 3-Way Handshake
2. TLS Handshake
3. 암호화된 HTTP 요청과 응답

[12-1] 1단계: TCP 연결 설정

클라이언트가 서버 443번 포트에 TCP 연결을 요청한다.



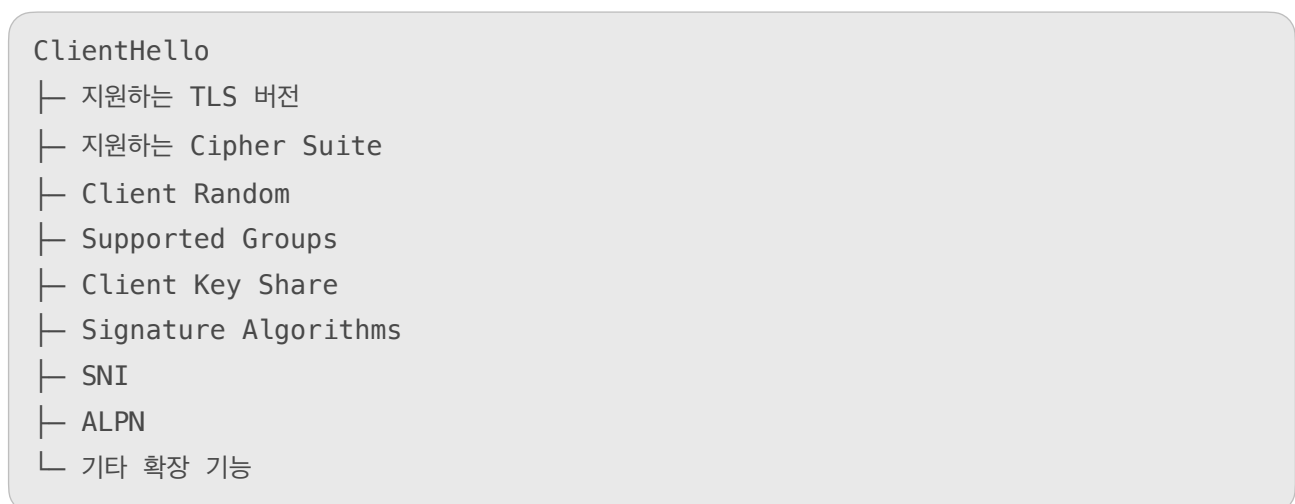
TCP 연결이 설정된 뒤 TLS Handshake가 시작된다.



HTTP/3에서는 TCP 연결 과정이 없다. UDP 기반 QUIC 연결 과정에 TLS 1.3 핸드셰이크가 통합된다.

[12-2] 2단계: ClientHello

클라이언트가 자신이 지원하는 TLS 설정과 키 합의에 필요한 정보를 전달한다.



[Cipher Suite (암호화 스위트, 사이퍼 스위트)]

어떤 암호 알고리즘 조합을 사용할 수 있는지 전달한다.

TLS 1.3의 예:

```
TLS_AES_128_GCM_SHA256
```

이는 대략 다음을 의미한다.

```
AES-128-GCM → 실제 데이터 암호화와 무결성 검증  
SHA-256      → 키 생성과 핸드셰이크 해시 등에 사용
```

TLS 1.3에서는 키 교환과 인증 알고리즘이 Cipher Suite 이름과 분리되어 협상된다.

[Key Share]

클라이언트가 ECDHE 키 합의에 사용할 임시 공개값을 전달한다.

[SNI]

SNI(Server Name Indication)는 클라이언트가 접속하려는 호스트 이름을 서버에 알려주는 TLS 확장 기능이다.

```
example.com  
api.example.com  
shop.example.com
```

하나의 IP 주소에서 여러 HTTPS 사이트를 제공하는 서버가 적절한 인증서를 선택하는 데 사용한다.
일반적인 TLS에서는 SNI가 평문으로 노출될 수 있다. ECH(Encrypted ClientHello)를 사용하면 이를 암호화할 수 있지만 모든 환경에서 항상 사용되는 것은 아니다.

[ALPN]

ALPN(Application-Layer Protocol Negotiation)은 TLS 위에서 어떤 애플리케이션 프로토콜을 사용할지 협상한다.

```
h2          → HTTP/2  
http/1.1    → HTTP/1.1
```

[12-3] 3단계: ServerHello

서버는 ClientHello의 선택지를 확인하고 사용할 TLS 설정을 결정한다.

```
ServerHello  
├─ 선택한 TLS 버전  
├─ 선택한 Cipher Suite  
├─ Server Random  
└─ Server Key Share
```

서버도 ECDHE에 사용할 임시 키 쌍을 생성한다.

서버 임시 개인키 b → 외부에 공개하지 않음
서버 임시 공개키 B → Server Key Share로 전송

클라이언트와 서버는 공개키인 Key Share만 교환한다.

클라이언트 → Client Key Share A → 서버
클라이언트 ← Server Key Share B ← 서버

각자 자신의 임시 개인키와 상대방의 임시 공개키로 동일한 공유 비밀을 계산한다.

[클라이언트]

클라이언트 개인키 a + 서버 공개키 B
→ ECDHE Shared Secret 계산

[서버]

서버 개인키 b + 클라이언트 공개키 A
→ 동일한 ECDHE Shared Secret 계산

여기서 '+'는 실제 덧셈이 아니라, 자신의 개인키와 상대방의 공개키를 이용한 타원곡선 연산을 의미하는 개념적 표현이다.

공유 비밀은 네트워크로 전송되지 않는다. 양쪽이 각자 계산하여 같은 결과를 얻는다.

[12-4] 4단계 핸드셰이크 암호키 생성

ECDHE(Elliptic Curve Diffie-Hellman Ephemeral)로 계산한 공유 비밀을 그대로 AES 키로 사용하는 것은 아니다.

TLS 1.3은 ECDHE 공유 비밀과 지금까지의 핸드셰이크 해시 등을 HKDF 기반 키 스케줄에 입력하여 여러 키와 비밀값을 파생한다.

개념적으로 표현하면 다음과 같다.

ECDHE 공유 비밀값 + ClientHello와 ServerHello의 해시

↓

HKDF 키 파생

↓

클라이언트 핸드셰이크 키

서버 핸드셰이크 키

각 방향의 IV와 관련 비밀값

핸드셰이크 키도 방향별로 다르다.

클라이언트 → 서버용 핸드셰이크 키
서버 → 클라이언트용 핸드셰이크 키

TLS 1.3에서는 “ServerHello” 이후의 주요 핸드셰이크 메시지가 이 키들로 보호된다.
따라서 네트워크에서 관찰할 수 있는 범위를 단순화하면 다음과 같다.

ClientHello → 일반적으로 관찰 가능
ServerHello → 일반적으로 관찰 가능
그 이후 주요 핸드셰이크 메시지 → 핸드셰이크 키로 암호화

[12-5] 5단계: EncryptedExtensions

서버가 추가로 선택하거나 적용한 확장 설정을 암호화하여 전달한다.

EncryptedExtensions
├─ 선택한 ALPN
├─ 서버가 적용할 TLS 확장 설정
└─ 기타 협상 결과

예를 들어 HTTP/2를 선택했다면 다음과 같은 결과가 포함될 수 있다.

ALPN: h2

HTTP/3의 QUIC 연결이라면 다음과 같다.

ALPN: h3

[12-6] 6단계: 서버 인증서 전달

서버는 자신의 신원을 증명하기 위해 인증서 체인을 전달한다.

Certificate
├─ public-insight.co.kr 서버 인증서
│ └─ 서버 공개키 포함
└─ 중간 CA 인증서

루트 CA 인증서는 보통 전송하지 않는다. 브라우저나 운영체제의 신뢰 저장소에 이미 있기 때문이다.

브라우저는 다음 항목을 확인한다.

1. 인증서의 도메인이 접속한 도메인과 일치하는가?
2. 인증서가 유효기간 내에 있는가?
3. 중간 CA의 서명이 정상인가?
4. 중간 CA가 신뢰하는 루트 CA로 연결되는가?
5. 인증서가 폐기되지 않았는가?

검증 구조는 다음과 같다.

서버 인증서
↓ 중간 CA 공개키로 서명 검증
중간 CA 인증서
↓ 루트 CA 공개키로 서명 검증
루트 CA 인증서
↓ 브라우저 신뢰 저장소에서 확인
신뢰 완료

이 과정을 통해 브라우저는 다음을 확인한다.

인증서에 포함된 서버 공개키가 CA에 의해 public-insight.co.kr에 대해 발급된 공개키다.

하지만 이 검증만으로 현재 접속한 상대방이 그 공개키와 짝인 개인키까지 가지고 있는지 아직 별도로 확인해야 한다.

[12-7] 7단계: CertificateVerify

인증서는 공개 정보이므로 공격자도 복사할 수 있다. 공격자가 서버 인증서만 복사했다고 해서 진짜 서버가 되어서는 안된다.

서버는 지금까지 주고받은 핸드셰이크 내용 서버 개인키로 전자서명하여 'CertificateVerify' 메시지를 만든다.

ClientHello부터 현재까지의 핸드셰이크 내용
↓ 해시
핸드셰이크 해시
↓ 서버 개인키로 서명
CertificateVerify

클라이언트는 서버 인증서 안에 들어 있는 서버 공개키로 서명을 검증한다.

CertificateVerify
↓ 서버 공개키로 검증
서버의 개인키 소유 여부 확인

두 종류의 서명을 정확히 구분해야 한다.

[인증서에 대한 CA 서명]

검증 키: 중간 CA 공개키

목적: 도메인과 서버 공개키의 연결을 CA가 인증했는지 확인

[CertificateVerify에 대한 서버 서명]

검증 키: 서버 인증서에 들어 있는 서버 공개키

목적: 현재 서버가 서버 개인키를 실제로 보유했는지 확인

공격자가 인증서 파일만 복사했다면 서버 개인키가 없으므로 올바른 CertificateVerify 서명을 만들 수 없다.

[12-8] 8단계: 서버 Finished

서버는 지금까지의 핸드셰이크가 변조되지 않았고 자신이 올바른 핸드셰이크 키를 계산했다는 사실을 증명하기 위해 'Finished' 메시지를 보낸다.

개념적으로 다음과 같이 계산한다.

지금까지의 전체 Handshake Hash

+ 서버 Finished Key

↓

서버 Finished 검증값

클라이언트도 자신이 계산한 핸드셰이크 키와 동일한 핸드셰이크 메시지를 이용해 예상 값을 계산한다.

받은 Finished == 클라이언트가 계산한 예상 Finished

→ 서버가 같은 키를 계산함

→ 핸드셰이크 내용이 변조되지 않음

값이 다름

→ 키가 다르거나 핸드셰이크가 변조됨

→ 연결 실패

CertificateVerify와 Finished도 역할이 다르다.

CertificateVerify

→ 서버 개인키 보유와 서버 인증을 확인

Finished

→ 핸드셰이크 전체 무결성과 키 계산의 일치를 확인

[12-9] 9단계: 클라이언트 Finished

클라이언트는 인증서, CertificateVerify, 서버 Finished 검증을 마친 뒤 자신의 Finished 메시지를 보낸다.



서버도 클라이언트 Finished를 검증한다. 이를 통해 클라이언트도 같은 핸드셰이크 키와 핸드셰이크 내용을 가지고 있음을 확인한다.

일반적인 HTTPS에서는 서버만 인증서를 제시한다. 서버가 클라이언트 인증서까지 요구하는 mTLS(Mutual TLS)에서는 클라이언트도 인증서와 CertificateVerify를 보낸다.

[12-10] 10단계: 애플리케이션 데이터용 트래픽 키 생성

양쪽은 실제 HTTP 데이터를 보호할 애플리케이션 트래픽 키를 TLS 1.3 키 스케줄에서 파생한다.

개념적으로 다음과 같이 이해할 수 있다.

ECDHE Shared Secret + 전체 Handshake Hash
↓ HKDF 기반 키 파생
애플리케이션 트래픽 키와 IV 생성

이 역시 방향별로 분리된다.

클라이언트 → 서버 암호화 키와 IV
서버 → 클라이언트 암호화 키와 IV

따라서 클라이언트가 서버에 보내는 데이터와 서버가 클라이언트에 보내는 데이터는 서로 다른 키로 보호된다.

TLS에서는 단순히 `세션키 하나`만 만드는 것이 아니라 다음처럼 용도별·방향별로 여러 비밀값과 키를 파생하고 이해하는 것이 정확하다.

핸드셰이크 보호용 키
애플리케이션 데이터 보호용 키
Finished 계산용 키
세션 재개용 비밀값
각 방향의 키와 IV

[12-11] 11단계: 암호화된 HTTP 통신

핸드셰이크가 완료되면 실제 HTTP 요청과 응답을 애플리케이션 트래픽 키로 암호화한다.

평문 HTTP 요청의 예는 다음과 같다.

클라이언트의 HTTP 요청

GET / HTTP/1.1

Host: public-insight.co.kr

Cookie: session_id=abc123

클라이언트 측 처리 과정은 다음과 같다.

HTTP 요청

↓ TLS가 클라이언트→서버 트래픽 키로 암호화

TLS Application Data

↓

TCP 세그먼트

↓

IP 패킷

서버는 대응하는 클라이언트→서버 트래픽 키로 복호화한 후 HTTP 요청을 처리한다.

응답은 반대 방향으로 처리한다.

서버의 HTTP 응답

↓ 서버→클라이언트 트래픽 키로 암호화

TLS Application Data

↓ 네트워크 전송

클라이언트가 서버→클라이언트 트래픽 키로 복호화

[전체과정]

TCP 3-way handshake
↓
ClientHello + Client Key Share
↓
ServerHello + Server Key Share
↓
ECDHE 공유 비밀 계산
↓
핸드셰이크 암호화 키 생성
↓
EncryptedExtensions
↓
서버 인증서 검증
↓
CertificateVerify로 서버 개인키 소유 확인
↓
서버 Finished
↓
클라이언트 Finished
↓
애플리케이션 트래픽 키 생성
↓
암호화된 HTTP 요청·응답

[13] [ECDHE 핵심 원리]

ECDHE는 Elliptic Curve Diffie-Hellman Ephemeral의 약자다.

- Elliptic Curve: 타원곡선 암호 기술 사용
 - Diffie-Hellman: 비밀값을 직접 보내지 않고 공유 비밀 합의
 - Ephemeral: 연결마다 임시 키 사용
-

[13-1] 임시 키 쌍 생성

클라이언트와 서버는 새로운 TLS 연결을 위해 임시 개인키와 공개키를 생성한다.

클라이언트

└─ 임시 개인값 a

└─ 임시 공개값 A

서버

└─ 임시 개인값 b

└─ 임시 공개값 B

공개된 타원곡선의 기준점 G 를 이용해 공개키를 만든다.

클라이언트 공개키 $A = aG$

서버 공개키 $B = bG$

여기서 ' aG ', ' bG '는 일반 정수 곱셈이 아니라 타원곡선 위의 스칼라 곱셈 연산이다.

[13-2] 공개키 교환

네트워크에는 공개키인 Key Share만 보낸다.

클라이언트 → 공개키 $A(aG)$ → 서버

클라이언트 ← 공개키 $B(bG)$ ← 서버

임시 개인키 a 와 b 는 절대로 네트워크로 전송하지 않는다.

[13-3] 공유 비밀값 계산

클라이언트는 자신의 개인키 ' a '와 서버의 공개키 ' B '를 이용한다.

aB

$= a(bG)$

$= abG$

서버는 자신의 개인키 ' b '와 클라이언트의 공개키 ' A '를 이용한다.

bA

$= b(aG)$

$= abG$

따라서 양쪽은 동일한 공유 비밀값을 얻는다.

클라이언트 Shared Secret $= abG$

서버 Shared Secret = abG

` $abG$ ` 자체를 네트워크로 전송한 것이 아니라 양쪽이 각자 계산했다는 점이 중요하다.

[13-4] 공격자가 계산하기 어려운 이유

공격자는 네트워크를 감청하여 다음 값을 확인할 수 있다.

공개된 기준점 G

클라이언트 공개키 $A = aG$

서버 공개키 $B = bG$

하지만 ` $aG$ `에서 개인키 ` $a$ `를 계산하거나 ` $bG$ `에서 개인키 ` $b$ `를 계산하려면 타원곡선 이산로그 문제를 풀어야 한다.

안전한 곡선과 충분한 키 크기를 사용하면 현실적인 시간 안에 계산하기 어렵다.

따라서 공격자는 두 공개키를 모두 알고 있어도 공유 비밀 ` $abG$ `를 쉽게 계산할 수 없다.

[13-5] 공유 비밀과 실제 암호키의 차이

ECDHE가 만든 ` $abG$ `는 최종 AES 키 그 자체가 아니다.

ECDHE 결과 abG

→ 공유 비밀 재료

공유 비밀 + 핸드셰이크 해시

→ HKDF 키 스케줄

→ 실제 핸드셰이크 키와 애플리케이션 트래픽 키

따라서 정확한 표현은 다음과 같다.

ECDHE로 양쪽이 같은 공유 비밀을 만들고, TLS 1.3은 그 공유 비밀과 핸드셰이크 정보를 HKDF에 넣어 실제 암호화 키들을 파생한다.

[14] [TLS 핸드셰이크는 언제 수행되는가?]

새로운 TLS 연결이 만들어질 때 핸드셰이크와 키 합의가 수행된다.

HTTP 요청 하나를 보낼 때마다 전체 핸드셰이크를 다시 하는 것은 아니다.

```
웹사이트 접속
↓
TCP 연결 생성
↓
TLS 핸드셰이크
├─ ClientHello
├─ ServerHello
├─ ECDHE 키 합의
├─ 서버 인증
└─ 암호키 생성
↓
하나의 TLS 연결에서 여러 HTTP 요청·응답 처리
```

예를 들어, 한 페이지에 다음 파일들이 있다고 한다.

```
index.html
style.css
app.js
logo.png
photo.jpg
```

각 파일 요청마다 ECDHE를 다시 수행하는 것은 아니다. 같은 TLS 연결을 재사용할 수 있다면 하나의 연결에서 여러 요청과 응답을 처리한다.

[14-1] HTTP/1.1의 경우

Keep-Alive를 사용하면 하나의 TCP/TLS 연결을 재사용할 수 있다.

```
TCP/TLS 연결 1개
├─ GET /index.html
├─ GET /style.css
├─ GET /app.js
└─ GET /logo.png
```

다만 HTTP/1.1은 병렬성을 확보하기 위해 브라우저가 같은 서버에 여러 TCP 연결을 만들기도 한다.

```
연결 1 → TCP 연결 + TLS 핸드셰이크
연결 2 → TCP 연결 + TLS 핸드셰이크
연결 3 → TCP 연결 + TLS 핸드셰이크
```

각각은 별개의 TLS 연결이므로 별도의 키를 사용한다.

[14-2] HTTP/2

HTTP/2는 하나의 TCP/TLS 연결에서 여러 요청과 응답을 독립적인 스트림으로 동시에 처리하는 멀티플렉싱을 지원한다.

TCP/TLS 연결 1개

- └ Stream 1: index.html
- └ Stream 3: style.css
- └ Stream 5: app.js
- └ Stream 7: image.png

따라서 일반적으로 HTTP/1.1보다 필요한 TCP/TLS 연결 수와 핸드셰이크 횟수를 줄일 수 있다.

[14-3] HTTP/3

HTTP/3는 TCP 대신 QUIC 연결을 사용한다.

QUIC 연결 1개

- └ TLS 1.3 보안 설정 통합
- └ Stream A: HTML
- └ Stream B: CSS
- └ Stream C: JavaScript
- └ Stream D: 이미지

새 QUIC 연결에서는 TLS 1.3 기반 키 합의가 이루어지지만 TCP 3-way handshake는 없다.

[14-4] 탭을 닫았다가 다시 들어가면?

기존 연결이 살아 있고 브라우저가 재사용할 수 있다면 새로운 TLS 핸드셰이크가 필요하지 않을 수 있다.

기존 연결이 종료되었다면 새 연결이 필요하며 다음 중 하나가 수행될 수 있다.

새 TLS 연결

- └ 전체 TLS 1.3 핸드셰이크
- └ 이전 정보를 활용한 세션 재개

세션 재개를 사용해도 새로운 연결에서 사용할 트래픽 키는 새로 파생된다.

[15] [Finished 메시지의 정확한 역할]

Finished는 지금까지의 핸드셰이크 메시지 전체가 변조되지 않았고, 상대방이 올바른 핸드셰이크 키를 계산했다는 것을 확인한다.

서버 Finished

- 서버가 올바른 키를 계산했음을 클라이언트가 확인
- 서버가 본 핸드셰이크 내용과 클라이언트가 본 내용이 일치하는지 확인

클라이언트 Finished

- 클라이언트가 올바른 키를 계산했음을 서버가 확인
- 양쪽의 핸드셰이크 내용이 일치하는지 확인

CertificateVerify와 Finished를 다시 비교하면 다음과 같다.

메시지	검증에 사용하는 값	주요 목적
CertificateVerify	서버 인증서의 공개키로 전자서명 검증	서버가 인증서에 대응하는 개인키를 실제로 보유했는지 확인하고 서버 신원을 인증
Finished	핸드셰이크 키와 전체 핸드셰이크 메시지의 해시 값	양쪽이 동일한 키를 생성했는지 확인하고 핸드셰이크 메시지가 변조되지 않았는지 검증

Finished 검증이 성공하면 양쪽은 암호화된 애플리케이션 데이터 통신을 안전하게 진행할 수 있다.

[16] [HTTP 데이터 암호화와 AEAD]

[16-1] 실제 암호화 흐름

평문 HTTP 요청의 예는 다음과 같다.

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/json
Cookie: session_id=abc123

{"id":"myeongsu","password":"1234"}
```

TLS가 이를 암호화하면 네트워크에서는 사람이 직접 읽을 수 없는 암호문으로 보인다.

```
17 03 03 00 a4 8f 29 c1 73 ...
```

서버는 합의된 대칭키로 복호화한 후 HTTP 요청을 처리한다.

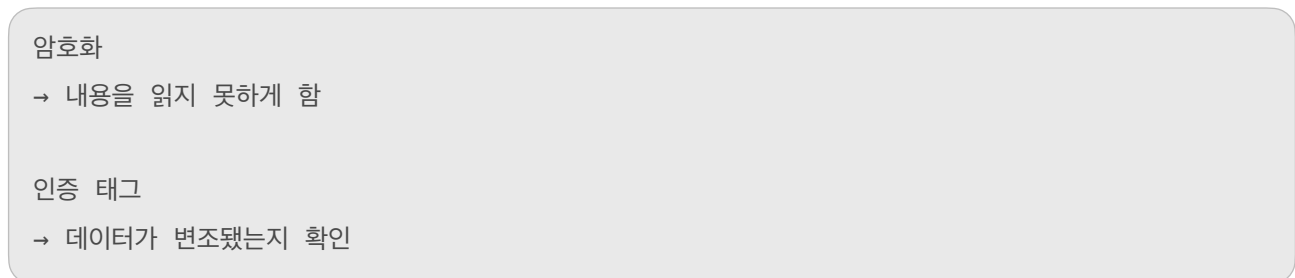


응답도 반대 방향으로 동일하게 처리된다.

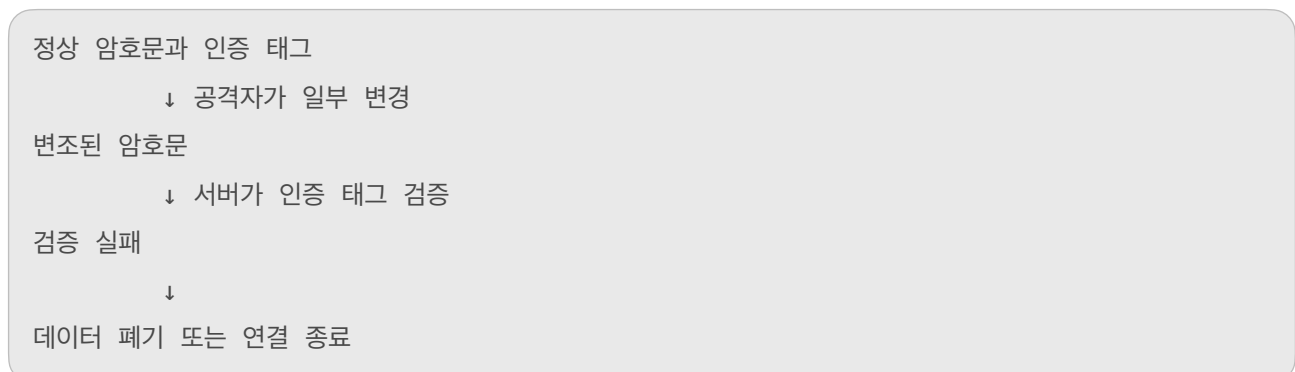
[16-2] AEAD

TLS 1.3은 AES-GCM이나 ChaCha20-Poly1305 같은 AEAD 방식을 사용한다.

AEAD(Authenticated Encryption with Associated Data)는 암호화와 무결성 검증을 함께 제공한다.



공격자가 암호문의 일부를 임의로 변경하면 서버의 무결성 검증에 실패한다.



따라서 HTTPS는 통신 내용을 숨기는 것뿐 아니라 전송 중 변조도 탐지한다.

[17] [TLS 1.2의 RSA 키 교환과 TLS 1.3의 차이]

TLS를 설명할 때 흔히 다음과 같이 말한다.

클라이언트가 대칭키를 서버 공개키로 암호화하고,
서버가 개인키로 복호화한다.

이 설명은 과거 TLS 1.2 이하에서 사용할 수 있었던 RSA 키 교환을 단순화한 설명이다.

[17-1] 과거 RSA 키 교환

클라이언트가 비밀값을 생성
↓ 서버 RSA 공개키로 암호화
서버로 전송
↓ 서버 RSA 개인키로 복호화
양쪽이 비밀값을 이용해 대칭키 생성

이 방식에서는 서버의 RSA 개인키가 나중에 유출될 경우, 공격자가 과거에 캡처해 둔 키 교환 데이터를 복호화할 위험이 있었다.

[17-1] TLS 1.3의 일반적인 방식

TLS 1.3에서는 RSA 키 교환을 제거했다.

클라이언트와 서버가 ECDHE 공개키 교환
↓
각자 동일한 공유 비밀 계산
↓
HKDF로 대칭키 파생

서버 인증서의 개인키
↓
CertificateVerify 전자서명에 사용

즉, 서버 인증서의 공개키와 개인키는 서버 인증에 사용되고, 실제 공유 비밀은 ECDHE로 만든다.

현대 HTTPS를 설명할 때는 다음과 같이 말하는 것이 정확하다.

인증서의 공개키와 서버 개인키는 서버를 인증하는 데 사용하고, 클라이언트와 서버는 ECDHE로 공유 비밀을 합의한다. TLS 1.3은 이 공유 비밀에서 대칭키를 파생하여 실제 HTTP 데이터를 암호화한다.

[18] [Forward Secrecy]

ECDHE의 마지막 `E`는 Ephemeral, 즉 임시라는 뜻이다.
클라이언트와 서버는 연결마다 새로운 임시 키를 만든다.

TLS 연결 1 → 임시 키 쌍 1 → 트래픽 키 1

TLS 연결 2 → 임시 키 쌍 2 → 트래픽 키 2

TLS 연결 3 → 임시 키 쌍 3 → 트래픽 키 3

서버 인증서의 장기 개인키는 서버 인증용 서명을 만드는 데 사용하고, 과거 연결의 ECDHE 임시 개인키는 연결 이후 폐기한다.

따라서 공격자가 나중에 서버의 장기 개인키를 탈취하더라도 과거에 캡처한 통신의 ECDHE 공유 비밀을 바로 계산하기 어렵다.

이 성질을 Forward Secrecy라고 한다.

서버 장기 개인키가 나중에 유출됨

↓

과거 ECDHE 임시 개인키는 이미 폐기됨

↓

과거 세션의 공유 비밀과 트래픽 키 계산이 어려움

이는 과거 RSA 키 교환과 비교했을 때 중요한 보안 향상이다.

[19] [HTTPS가 보호하는 정보와 노출되는 정보]

[19-1] TLS가 보호하는 HTTP 정보

TLS Handshake가 완료된 뒤 다음 HTTP 정보는 암호화된다.

- HTTP Method
- URL 경로
- Query Parameter
- HTTP Header
- Cookie
- Authorization 헤더
- 요청 Body
- 응답 Body
- Status Code를 포함한 HTTP 응답 내용

예를 들어 다음 URL에 접속한다고 하자.

`https://example.com/users/10?token=abc`

TLS 연결이 설정된 후 HTTP 요청에 들어가는 다음 부분은 암호화된다.

`/users/10?token=abc`

[19-2] 일반적으로 관찰될 수 있는 정보

HTTPS가 모든 네트워크 메타데이터를 숨기는 것은 아니다.

다음 정보는 라우팅과 전송 또는 트래픽 관찰을 통해 노출될 수 있다.

- 출발지 IP 주소
- 목적지 IP 주소
- 포트 번호
- 패킷 크기
- 통신 시점
- 통신량과 지속 시간
- 일반적인 TLS의 SNI 호스트 이름

공격자는 다음 정도를 관찰할 수 있다.

“이 사용자가 특정 IP의 서버 443번 포트와 통신하고 있다.”

그러나 정상적인 TLS 통신을 복호화하지 못한다면 다음 내용은 직접 알기 어렵다.

“어떤 URL 경로를 요청했는가?”

“어떤 쿠키와 비밀번호를 보냈는가?”

“서버가 어떤 응답 내용을 반환했는가?”

ECH와 암호화 DNS 등을 함께 사용하면 일부 노출 정보를 추가로 줄일 수 있다.

[19] [HTTPS와 중간자 공격]

중간자 공격자는 클라이언트와 서버 사이에서 데이터를 가로채거나 변조하려 한다.

클라이언트 ↔ 공격자 ↔ 서버

서버 인증 과정이 없다면 공격자가 서버인 것처럼 행동하며 자신의 공개키를 전달할 수 있다.

HTTPS에서는 브라우저가 서버 인증서와 CertificateVerify를 검증한다.

공격자가 가짜 인증서 전달

↓

신뢰할 수 있는 CA 서명 없음

또는 도메인 불일치

↓

브라우저 인증서 검증 실패

↓

보안 경고

공격자가 진짜 서버 인증서를 복사하더라도 서버 개인키가 없다면 올바른 CertificateVerify를 만들 수 없다.

단, 사용자가 인증서 경고를 무시하고 강제로 접속하거나 공격자가 사용자 시스템에 신뢰받는 루트 인증서를 설치할 수 있다면 서버 인증 보호가 약화될 수 있다.

[20] [Self-Signed 인증서]

Self-Signed 인증서는 공개적으로 신뢰받는 CA가 아니라 인증서 소유자가 자신의 개인키로 직접 서명한 인증서다.

서버 인증서

└ 서버 또는 조직 자신이 직접 서명

Self-Signed 인증서도 TLS 암호화 자체는 가능하다. 그러나 브라우저는 서명자를 기본적으로 신뢰하지 않기 때문에 인증서 경고가 발생한다.

주로 다음 환경에서 사용한다.

- 로컬 개발
- 테스트 서버
- 내부 네트워크
- 사설 PKI
- 인증서를 직접 배포하여 신뢰 관계를 설정한 시스템

공개 웹사이트 운영 환경에서는 일반적으로 브라우저와 운영체제가 신뢰하는 CA가 발급한 인증서를 사용한다. Self-Signed 인증서가 무조건 암호학적으로 약하다는 뜻은 아니다. 핵심 문제는 클라이언트가 해당 서명자를 신뢰할 근거가 기본적으로 없다는 것이다.

[21] [TLS 세션 재개]

매번 새 연결에서 인증서 전달과 전체 키 합의를 반복하면 네트워크 왕복과 연산 비용이 발생한다.

TLS 1.3은 이전 연결 정보를 활용해 다음 연결의 핸드셰이크 비용을 줄이는 세션 재개 기능을 제공한다.

[21-1] Session Ticket과 PSK

첫 연결에서 서버는 이후 세션 재개에 사용할 정보를 클라이언트에게 제공할 수 있다.

[첫 번째 연결]

전체 TLS 1.3 핸드셰이크

↓

서버가 NewSessionTicket 제공

↓

클라이언트가 세션 티켓과 관련 PSK 정보 보관

다음 연결에서는 클라이언트가 이전 정보를 활용할 수 있다.

[다음 연결]

ClientHello에 PSK 식별 정보 제공

↓

서버가 세션 재개 허용

↓

전체 인증서 기반 핸드셰이크보다 비용 감소

세션 재개를 사용해도 새 연결에서 실제로 사용할 트래픽 키는 새로 파생된다.

설정에 따라 PSK와 새로운 ECDHE 공유 비밀을 함께 사용하는 `PSK + ECDHE` 방식도 사용할 수 있다. 이 방식은 세션 재개의 효율성과 새로운 ECDHE 키 합의의 Forward Secrecy 특성을 함께 활용할 수 있다.

[21-2] 0-RTT

TLS 1.3은 조건에 따라 세션 재개 시 첫 왕복이 끝나기 전에 Early Data를 전송하는 0-RTT 기능을 사용할 수 있다.

ClientHello + 암호화된 Early Data

→ 서버

하지만 0-RTT 데이터는 재전송 공격에 노출될 수 있다. 공격자가 암호문을 해독하지 못하더라도 동일한 0-RTT 요청을 다시 전달할 가능성이 있다.

상대적으로 사용을 고려할 수 있는 요청:

GET /products

GET /news

주의해야 하는 요청:

POST /payment

POST /order

POST /transfer

조회처럼 멍등적인 요청과 달리 결제·주문·송금은 같은 요청이 반복되면 상태가 중복 변경될 수 있으므로 0-RTT 사용에 주의해야 한다.

[22] [HTTPS 전체 과정]

HTTP/1.1 또는 HTTP/2 기반 HTTPS 통신의 전체 과정을 네트워크 시작부터 정리하면 다음과 같다.

1. DNS 조회
example.com → 서버 IP 확인
2. TCP 연결 설정
3-Way Handshake
3. TLS ClientHello
TLS 버전, Cipher Suite, Key Share 전달
4. TLS ServerHello
사용할 TLS 설정과 Server Key Share 선택
5. 서버 인증서 전달
인증서 체인과 서버 공개키 전달
6. 인증서 검증
CA 서명, 도메인, 유효 기간, 용도 확인
7. 서버 개인키 보유 확인
CertificateVerify 전자서명 검증
8. ECDHE 키 합의
동일한 공유 비밀값 계산
9. 대칭키 생성
공유 비밀값으로 통신용 키 파생
10. Finished 교환
Handshake 변조 여부와 키 일치 확인
11. 암호화된 HTTP 통신
대칭키로 요청과 응답 암호화
12. 연결 종료 또는 재사용
Keep-Alive로 연결을 재사용할 수 있음

간단히 줄이면 다음과 같다.

```
DNS
↓
TCP Handshake
↓
TLS Handshake
↓
서버 인증
↓
대칭키 생성
↓
암호화된 HTTP 통신
```

[23] [서버에서는 누가 TLS를 처리하는가?]

일반적인 웹 서버 구성에서는 Nginx, Apache, Caddy, Envoy 또는 로드 밸런서가 TLS 처리를 담당할 수 있다.

예를 들어 다음 구조라면 Nginx가 TLS 종단점이다.

```
브라우저
↓ HTTPS
Nginx
├─ 인증서와 개인키 사용
├─ TLS 핸드셰이크 수행
├─ HTTP 메시지 복호화
└─ 백엔드로 요청 전달
↓
Go / Spring / FastAPI 백엔드
```

Nginx가 일반적으로 수행하는 작업은 다음과 같다.

- TCP 443번 포트에서 연결 수락
- ClientHello 처리
- SNI에 따라 인증서 선택
- ALPN으로 HTTP/2 또는 HTTP/1.1 협상
- ServerHello와 ECDHE 처리
- 인증서와 CertificateVerify 전달
- TLS 레코드 암호화·복호화
- 복호화된 HTTP 요청을 백엔드로 전달

실제 암호 연산은 Nginx가 사용하는 OpenSSL 등의 TLS 라이브러리가 담당한다.

CDN이나 클라우드 로드 밸런서를 사용하면 TLS가 원본 서버가 아니라 CDN 또는 로드 밸런서에서 종료될 수도 있다.

브라우저 ← HTTPS → CDN/Load Balancer ← HTTPS 또는 HTTP → 원본 서버

어디에서 TLS가 종료되는지에 따라 인증서와 개인키를 보관하는 위치도 달라진다.

[24] [명령어로 TLS 확인하기]

[24-1] curl

TLS 연결 과정을 간단히 확인하려면 다음 명령을 사용한다.

```
curl -v https://example.com
```

환경과 curl 빌드에 따라 다음 정보를 확인할 수 있다.

- 연결한 IP와 포트
- TLS 버전
- Cipher Suite
- ALPN 협상 결과
- 서버 인증서 Subject
- 서버 인증서 Issuer
- 인증서 검증 결과
- HTTP 버전
- HTTP 상태 코드

특정 HTTP 버전을 시도하려면 curl 빌드가 지원하는 범위에서 다음 옵션을 사용할 수 있다.

```
curl --http1.1 -v https://example.com
curl --http2 -v https://example.com
curl --http3 -v https://example.com
```

`--http3`는 설치된 curl이 HTTP/3와 QUIC을 지원하도록 빌드된 경우에만 사용할 수 있다.

[24-2] OpenSSL s_client

서버의 TLS 연결과 인증서 체인을 자세히 확인하려면 다음 명령을 사용한다.

```
openssl s_client \  
-connect example.com:443 \  
-servername example.com
```

`-servername`은 ClientHello의 SNI로 전달할 도메인을 지정한다. 한 IP에서 여러 HTTPS 사이트를 운영하는 서버에서는 올바른 인증서를 받기 위해 중요하다.

출력에서 다음 내용을 확인할 수 있다.

- Certificate Chain
- 서버 인증서 Subject
- 인증서 Issuer
- 인증서 유효기간
- 협상된 TLS 버전
- Cipher Suite
- ALPN 결과
- 인증서 검증 결과

TLS 1.3만 허용하여 테스트하려면 OpenSSL 버전에 따라 다음과 같이 실행할 수 있다.

```
openssl s_client \  
-connect example.com:443 \  
-servername example.com \  
-tls1_3
```

[24-3] 인증서 내용 확인

PEM 인증서 파일이 있다면 다음 명령으로 주요 정보를 확인할 수 있다.

```
openssl x509 \  
-in certificate.pem \  
-noout \  
-text
```

간단히 Subject, Issuer, 유효기간과 SAN을 확인하려면 다음 명령들을 사용할 수 있다.

```
openssl x509 -in certificate.pem -noout -subject -issuer -dates  
openssl x509 -in certificate.pem -noout -ext subjectAltName
```


[25] [핵심 개념 비교]

CA서명과 서버 서명

구분	CA의 인증서 서명	서버의 CertificateVerify 서명
서명자	인증서를 발급한 중간 CA	현재 접속한 서버
서명 대상	서버 인증서의 내용	현재까지 교환한 핸드셰이크 메시지의 해시값
사용 개인키	중간 CA의 개인키	서버 인증서에 대응하는 서버 개인키
검증에 사용하는 키	중간 CA 인증서의 공개키	서버 인증서에 포함된 서버 공개키
검증 주체	클라이언트	클라이언트
주요 목적	도메인 정보와 서버 공개키의 연결을 인증	서버가 인증서에 대응하는 개인키를 실제로 보유했는지 증명
검증 결과	인증서가 신뢰할 수 있는 CA에 의해 발급되었는지 확인	현재 연결한 상대가 인증서의 실제 소유자인지 확인

ECDHE 공유 비밀과 트래픽 키

구분	생성 또는 구성	주요 역할
ECDHE 임시 개인키	클라이언트와 서버가 TLS 연결마다 각각 생성	외부에 공개하지 않고 공유 비밀값 계산에 사용
ECDHE Key Share	임시 개인키로 생성한 임시 공개키	상대방과 네트워크를 통해 교환
ECDHE Shared Secret	자신의 임시 개인키와 상대방의 Key Share로 계산	TLS 암호키 생성에 사용하는 공통 비밀 자료
Handshake Hash	지금까지 교환한 핸드셰이크 메시지를 해시	핸드셰이크 내용을 키 생성과 검증에 반영
HKDF	공유 비밀값과 핸드셰이크 해시 등을 입력받아 동작	용도와 통신 방향에 따라 여러 암호키 파생
Handshake Traffic Key	TLS 1.3 키 스케줄을 통해 생성	ServerHello 다음의 암호화된 핸드셰이크 메시지 보호
Application Traffic Key	핸드셰이크 완료 과정에서 별도로 파생	실제 HTTP 요청과 응답 보호

암호 기술별 역할

암호 기술	TLS에서의 역할	대표적인 사용 위치
대칭키 암호	같은 비밀키 계열을 이용해 데이터를 빠르게 암호화·복호화	핸드셰이크 메시지와 실제 HTTP 데이터 보호
공개키·개인키	서버 신원 인증과 전자서명 생성·검증	인증서와 CertificateVerify
ECDHE	개인키를 전송하지 않고 동일한 공유 비밀값 합의	ClientHello와 ServerHello의 Key Share 교환
해시 함수	데이터를 고정 길이 요약값으로 변환	핸드셰이크 해시, 전자서명, 키 파생
전자서명	개인키 보유자 확인과 서명 대상의 무결성 검증	CA의 인증서 서명과 서버의 CertificateVerify
HKDF	하나의 비밀 재료에서 용도·단계·방향별 키 파생	핸드셰이크 키와 애플리케이션 트래픽 키 생성
AEAD	데이터 암호화와 변조 탐지를 동시에 수행	TLS 레코드의 암호화 및 인증 태그 생성

